

Complete Git & GitHub Study Notes

Based on Abhas Paul's Complete Git and GitHub Tutorial

[Version Control](#) | [Branching](#) | [Pull Requests](#) | [Open Source Contribution](#)

1. Introduction

Git and GitHub are two of the most essential tools for modern software development. This tutorial covers everything from the absolute basics to advanced workflows used in real-world open-source contribution.

What is Git?

Git is a distributed version control system (VCS). It tracks every change made to your code over time, allowing you to revert to any earlier state, collaborate with others safely, and maintain a complete history of a project.

- Created by Linus Torvalds in 2005 (creator of the Linux kernel).
- Runs entirely on your local machine — no internet is required.
- Stores project history as a series of snapshots called commits.
- Free, open-source, and the industry standard.

What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories online. It adds collaboration features on top of Git.

- Acts as a remote backup for your code.
- Enables multiple developers to work on the same project.
- Provides tools for code review, issue tracking, and project management.
- The world's largest platform for open-source software.

NOTE: Git ≠ GitHub. Git is the tool; GitHub is a hosting service that uses Git. Alternatives to GitHub include GitLab and Bitbucket.

Why Use Git & GitHub?

- Track every change made to your code — nothing is ever truly lost.

- Collaborate with teams without overwriting each other's work.
 - Maintain multiple versions of a project using branches.
 - Contribute to open-source projects from anywhere in the world.
 - Easily undo mistakes by reverting to previous commits.
-

2. Setting Up Git

2.1 Downloading & Installing Git

Download Git from the official site: <https://git-scm.com/downloads>

- Windows users should use Git Bash for running all Git commands.
- macOS and Linux often have Git pre-installed; verify with:

```
git --version
```

2.2 First-Time Configuration

Before making any commits, you must tell Git your name and email. These details are permanently attached to every commit you make.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

The `--global` flag applies these settings to every repository on your machine. To apply settings to a single project only, omit `--global`.

2.3 Basic Linux / Terminal Commands

These commands are used frequently inside Git Bash or any terminal:

- **ls:** List all files and folders in the current directory.
 - **cd folder-name:** Change into a directory.
 - **cd ..:** Move up one directory level.
 - **pwd:** Print the current working directory path.
 - **mkdir folder-name:** Create a new folder.
 - **touch file.txt:** Create a new empty file.
 - **clear:** Clear the terminal screen.
-

3. Core Git Concepts & Commands

3.1 Initializing a Repository

A Git repository (repo) is a project folder that Git is tracking.

```
git init
```

Run this inside any folder to start tracking it with Git. It creates a hidden `.git` folder that stores all version history. Never delete this folder manually.

3.2 The Three-Stage Workflow

Every change in Git goes through three stages before it is permanently recorded:

Working Directory → **Staging Area** → **Repository (History)**

- **Working Directory:** Where you write and edit your files.
- **Staging Area (Index):** A preparation zone. You explicitly choose which changes to include in the next commit.
- **Repository:** The permanent history of all commits.

3.3 Checking the Status of Files

```
| git status
```

Shows which files are modified, staged, or untracked. Run this command frequently — it is your best tool for understanding what Git sees.

3.4 Staging Changes

Staging means selecting which changes will go into your next commit.

```
| git add filename.txt      # Stage a specific file
| git add .                 # Stage ALL changed files at once
```

NOTE: Staging gives you fine-grained control. You can modify ten files but only commit three of them.

3.5 Committing Changes

A commit is a permanent snapshot of your staged changes, saved to history.

```
| git commit -m "Your descriptive commit message"
```

Write commit messages in the present tense, describing what the commit does (e.g., 'Add login functionality', not 'Added login').

3.6 Viewing Project History

```
| git log                  # Detailed history of all commits
| git log --oneline        # Condensed one-line-per-commit view
```

Each commit has a unique SHA hash (a long string like a3f82bc...) that identifies it permanently.

3.7 Adding More Content to Files

After creating a file and making a commit, you can continue to edit it. Each new set of changes must go through the same add → commit cycle.

3.8 Unstaging Changes

If you staged a file by mistake and want to remove it from the staging area (without discarding your edits):

```
git restore --staged filename.txt
```

3.9 Removing (Undoing) a Commit

Two important methods exist for removing commits from history:

git reset (Soft)

Moves the commit pointer back but keeps changes in the staging area.

```
git reset --soft HEAD~1 # Undo last commit, keep changes staged
```

git reset (Hard)

Permanently discards a commit and ALL associated changes. Use with caution.

```
git reset --hard <commit-hash>
```

NOTE: HEAD refers to the most recent commit. HEAD~1 means 'one commit before HEAD'.

4. Stashing

Stashing is like putting your unfinished work into a temporary drawer so you can switch tasks without losing or committing incomplete changes.

4.1 When to Use Stash

- You are in the middle of a feature but need to urgently fix a bug on another branch.
- You need to pull the latest changes from remote but have local edits that would conflict.

4.2 Stash Commands

```
git stash # Save current changes to the stash
git stash pop # Re-apply the most recent stash and remove it
git stash clear # Delete all stashed changes permanently
git stash list # View all stashed entries
```

NOTE: git stash pop is different from git stash apply. 'pop' removes the stash entry after applying it; 'apply' keeps it.

5. Working with GitHub

5.1 Creating a Repository on GitHub

On GitHub, click 'New Repository', give it a name, choose public or private, and optionally add a README. Do not initialize with a README if you already have a local repo — it will cause conflicts.

5.2 Connecting Local Repo to Remote

A remote is a version of your repository hosted on GitHub. The standard name for the primary remote is origin.

```
git remote add origin https://github.com/username/repo-name.git
git remote -v          # Verify remote connections
```

5.3 Pushing Changes to GitHub

Pushing sends your local commits to the remote repository.

```
git push origin main      # Push local 'main' branch to GitHub
git push -u origin main    # Set upstream and push (first time)
```

The -u flag sets a tracking relationship so future pushes only need git push.

5.4 Pulling Changes from GitHub

```
git pull origin main      # Fetch and merge remote changes
```

6. Branching

6.1 What is a Branch?

A branch is an independent line of development. Think of the main branch as the production-ready trunk of a tree, and feature branches as smaller branches growing off it.

- The default branch is usually called main (or master in older repos).
- Every branch has its own isolated commit history.
- Changes on one branch do not affect other branches until merged.

6.2 Why Use Branches?

- Develop new features without breaking working code on main.
- Multiple developers can work independently on different features.
- Experiment freely — if something fails, simply delete the branch.
- Enforce code review before changes reach the main branch.

6.3 Branch Commands

```
git branch                # List all local branches
git branch feature-login   # Create a new branch
```

```
git switch feature-login      # Switch to that branch
git switch -c feature-login   # Create AND switch in one command
git branch -d feature-login   # Delete a branch (safe)
git branch -D feature-login   # Force-delete a branch
```

6.4 Merging Branches

To merge a feature branch back into main, first switch to main and then run merge:

```
git switch main
git merge feature-login
```

Git will automatically combine both histories. If the same lines were changed in both branches, a merge conflict occurs (covered in Section 9).

7. Forking, Cloning & Pull Requests

7.1 Forking

A fork is your personal copy of someone else's repository on GitHub. It lives in your GitHub account and is completely independent of the original.

- Click 'Fork' on any GitHub repository to create your own copy.
- You have full write access to your fork; you cannot push directly to the original.
- Forking is the standard starting point for open-source contributions.

7.2 Cloning

Cloning downloads a remote repository to your local machine.

```
git clone https://github.com/your-username/forked-repo.git
```

This creates a local folder containing the project with all its history, and automatically sets up origin to point to your fork.

7.3 Upstream Remote

When contributing to open source, you need to track the original project (not just your fork) so you can sync future changes. This original repo is called upstream.

```
git remote add upstream https://github.com/original-owner/repo.git
git remote -v                # Should now show both origin and upstream
```

7.4 What is a Pull Request (PR)?

A pull request is a formal request to merge your changes from your fork (or branch) into the original repository. It is not a Git command — it is a GitHub feature.

- Opens a discussion thread where maintainers can review your code.
- Reviewers can request changes, ask questions, or approve.

- Only after approval does a maintainer merge the PR.
- All professional and open-source collaboration uses this workflow.

7.5 Standard Open-Source Contribution Workflow

1. Fork the original repository on GitHub.
2. Clone your fork to your local machine.
3. Add the original repo as 'upstream' remote.
4. Create a new feature branch (NEVER work directly on main).
5. Make your changes, stage, and commit them.
6. Push your branch to your fork on GitHub.
7. Open a Pull Request from your branch to the original repo.
8. Address review feedback; sync with upstream if needed.

NOTE: NEVER commit directly on main when contributing to open-source. Always create a separate branch for every feature or bug fix.

8. Keeping Your Fork in Sync

The original repository may receive new commits while you work on your feature. You must keep your fork up to date to avoid conflicts.

```
git fetch upstream          # Download upstream changes without merging
git switch main             # Switch to your main branch
git merge upstream/main     # Merge upstream changes into your main
git push origin main        # Push updated main to your fork
```

After syncing, rebase your feature branch on the updated main to keep a clean history:

```
git switch feature-branch
git rebase main
```

9. Advanced Topics

9.1 Squashing Commits

Squashing combines multiple small commits into a single, cleaner commit. This is common practice before opening a pull request to keep the project history readable.

```
git rebase -i HEAD~3        # Interactively rebase the last 3 commits
```

In the interactive editor, change 'pick' to 'squash' (or 's') on the commits you want to combine. The first commit should remain as 'pick'.

9.2 Rebase

Rebasing moves your branch's commits onto a new base commit, creating a linear history. It is an alternative to merging.

```
git rebase main # Rebase current branch onto main
```

- Produces a cleaner, linear commit history.
- Rewrites commit history — do not rebase branches others are working on.
- Merge preserves the exact history; rebase rewrites it for clarity.

9.3 Force Push

After rebasing or squashing, your local branch history differs from the remote. A normal push will be rejected. You must force-push:

```
git push --force origin feature-branch
```

NOTE: Only force-push on your own feature branches. Never force-push to a shared branch like main — it can permanently delete other people's work.

9.4 git reset --hard

This command moves the branch pointer to a specific commit AND permanently discards all changes after that point in the working directory and staging area.

```
git reset --hard <commit-hash>
```

Use this when you want to completely discard a series of changes. This action cannot be easily undone.

10. Merge Conflicts

10.1 What Causes a Merge Conflict?

A merge conflict occurs when two branches modify the same line(s) of the same file differently. Git does not know which version to keep, so it stops and asks you to decide.

10.2 What a Conflict Looks Like

Git marks the conflicting section in the file like this:

```
<<<<<< HEAD
This is my local version of the line.
=====
This is the incoming version from the other branch.
>>>>>> feature-branch
```

10.3 Resolving a Merge Conflict

9. Open the conflicting file(s) in your editor.
10. Decide which version to keep (or write a combination).
11. Delete ALL the conflict markers (<<<<<<, =====, >>>>>>).
12. Stage the resolved file: git add filename.txt

13. Complete the merge with: git commit

NOTE: Modern code editors like VS Code display conflict markers visually and offer 'Accept Current', 'Accept Incoming', or 'Accept Both' buttons to simplify resolution.

11. Important Rules & Best Practices

Commit Hygiene

- Write clear, meaningful commit messages that describe the why, not just the what.
- Make small, frequent commits rather than one large commit at the end.
- Each commit should represent one logical unit of change.

Branching Rules

- Always create a new branch for every feature or bug fix.
- Never commit directly to main or master in a collaborative project.
- Delete feature branches after they are merged to keep the repo clean.

Remote Collaboration

- Always pull the latest changes (git pull) before starting new work.
- Review your changes carefully before pushing.
- Do not force-push to shared branches.

Open Source Rules

- Read the project's CONTRIBUTING.md file before making changes.
 - Keep your fork in sync with upstream regularly.
 - One pull request should address one concern — do not mix features and bug fixes.
-

12. Practical Examples

Example 1: Creating Your First Local Repository

```
mkdir my-project
cd my-project
git init
touch README.md
git add .
git commit -m "Initial commit: add README"
```

Example 2: Full Feature Branch Workflow

```
git switch -c feature/add-login      # Create and switch to new branch
# ... make your code changes ...
git add .
git commit -m "Add login page UI"
git push origin feature/add-login    # Push branch to GitHub
# Open a Pull Request on GitHub
# After PR is merged:
git switch main
git pull origin main                # Sync local main
git branch -d feature/add-login      # Delete the feature branch
```

Example 3: Syncing Your Fork Before a New Contribution

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
git switch -c fix/broken-button
```

Example 4: Squashing 3 Commits into 1

```
git rebase -i HEAD~3
```

In the editor that opens:

```
pick 1b2c3 Add button component
squash d4e5f6 Fix button styles
squash g7h8i9 Fix button spacing
```

Save, then write a single combined commit message when prompted.

13. Command Quick-Reference Cheat Sheet

Command	What it Does	Section
<code>git init</code>	Initialize a new Git repository	3.1
<code>git status</code>	Show status of files	3.3
<code>git add .</code>	Stage all changes	3.4
<code>git commit -m '...'</code>	Save staged changes to history	3.5
<code>git log --oneline</code>	View condensed commit history	3.6
<code>git restore --staged file</code>	Unstage a file	3.8
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes	3.9
<code>git reset --hard <hash></code>	Permanently discard commits	9.4
<code>git stash</code>	Save unfinished work temporarily	4.2

Command	What it Does	Section
<code>git stash pop</code>	Re-apply last stash	4.2
<code>git remote add origin <url></code>	Connect to a remote repo	5.2
<code>git push origin main</code>	Upload commits to GitHub	5.3
<code>git pull origin main</code>	Download & merge remote changes	5.4
<code>git branch <name></code>	Create a new branch	6.3
<code>git switch <name></code>	Switch to a branch	6.3
<code>git switch -c <name></code>	Create & switch to a branch	6.3
<code>git merge <branch></code>	Merge a branch into current	6.4
<code>git clone <url></code>	Clone a remote repo locally	7.2
<code>git remote add upstream <url></code>	Track the original repo	7.3
<code>git fetch upstream</code>	Download upstream changes	8
<code>git rebase main</code>	Rebase branch onto main	9.2
<code>git rebase -i HEAD~N</code>	Interactive rebase (squash, etc.)	9.1
<code>git push --force origin <branch></code>	Force push after rebase	9.3

14. Summary

Key Takeaways

- **Git** is a local version control system that tracks your code history through commits.
- **GitHub** is a cloud platform that hosts Git repositories and enables team collaboration.
- **The core workflow** is: Edit → git add → git commit → git push.
- **Branches** allow parallel development without affecting the main codebase.
- **Stashing** saves incomplete work temporarily so you can switch contexts.
- **Forking & PRs** are the foundation of open-source contribution — fork, branch, code, push, and open a PR.
- **Keeping in sync** with upstream prevents outdated code and difficult merge conflicts.
- **Squashing & rebasing** clean up history before merging into the main project.
- **Merge conflicts** occur when two branches edit the same lines — resolve them manually and re-commit.
- **Best practice:** Never commit directly to main. Always use feature branches.

— End of Notes —